ARS-Stack: Adaptive Resampler-Selection Stacking Ensemble for Imbalanced Classification

A Complete Academic Project Plan — Core ML, Semester Capstone

Prepared for: Single-Student Semester Project (2–3 Months)

July 13, 2026

Contents

1. Project Overview **3**

2. Problem Statement & Motivation **4**

3. Literature Review Summary **5**

4. Proposed System: ARS-Stack **10**
4.1 Core Idea 10
4.2 Architecture & MLOps Pipeline 10

5. Datasets **12**

6. Algorithms & Tools **13**
6.1 Resampling Techniques (candidate pool for the adaptive selector) 13
6.2 Clustering / Region Detection 13
6.3 Base Learners (Ensemble) 13
6.4 Meta-Learner 13
6.5 Evaluation Metrics 13
6.6 Core Libraries 13

7. MLOps & Orchestration Integration **14**
7.1 Components and Their Role 14
7.2 Why Prefect/Airflow Instead of Just Running Scripts 15
7.3 Suggested Repository Structure 15

8. Project Timeline (12 Weeks) **16**
8.1 Week-by-Week Breakdown 16

9. Evaluation Plan **18**

10. Risk Assessment & Mitigation **19**

11. Expected Contribution & Publication Angle **20**

12. Deliverables Checklist **21**

13. References (Literature Reviewed) 22

1. Project Overview

Title: Adaptive Resampler-Selection Framework with Stacking Ensemble for Imbalanced Classification (ARS-Stack)

Domain: Core Machine Learning — Class-Imbalance Handling / Ensemble Learning

Type of contribution: Combination + optimization of existing published methods into one novel, automated pipeline (not a from-scratch invention) — suitable for a research paper and strong for a semester capstone grade requiring novelty.

One-line pitch: Every existing hybrid-sampling + ensemble paper picks *one* resampling technique (SMOTE, ADASYN, SMOTE-Tomek, etc.) and applies it uniformly across the whole dataset. This project builds a lightweight decision layer that automatically chooses the *best-suited* resampling technique **per local region of the minority class**, then feeds the result into a stacking ensemble — and wraps the whole thing in a reproducible MLOps pipeline (versioning, orchestration, tracking, containerized serving).

Why this is gradeable as "novel":

- No paper reviewed in the literature survey performs *per-cluster adaptive resampler selection* combined with a *confidence-weighted stacking ensemble*.
- It directly answers a gap explicitly raised by prior work: the best resampler is dataset- and region-dependent, yet all existing pipelines pick one resampler manually and apply it globally.
- It is an engineering-feasible combination of known, well-documented building blocks — realistic for one person in 10–12 weeks.

2. Problem Statement & Motivation

Real-world classification problems (fraud detection, rare disease diagnosis, network intrusion, churn) are almost always imbalanced: the class you care about most is the rarest. Standard classifiers optimize for overall accuracy and therefore learn to ignore the minority class.

The standard fix is **resampling** (oversampling the minority class, undersampling the majority class, or both) followed by **ensemble learning** (combining several classifiers to improve robustness). The literature review below shows that:

1. Resampling technique performance is highly inconsistent across datasets and even across regions of the same dataset.
2. Almost every hybrid method picks one resampler and hard-codes it.
3. Ensemble weighting is usually static, not adapted to how reliable each base learner was on a given resampled subset.

Research question: *Can automatically selecting the resampling strategy per local minority-class neighborhood, combined with a confidence-weighted stacking ensemble, outperform fixed-resampler baselines across multiple imbalance domains (binary extreme imbalance, binary moderate imbalance, and multiclass imbalance)?*

3. Literature Review Summary

S.No	Topic	Algorithm	Dataset	Method	Approach	Output / Outcome	Improvement Needed / Research Gap
1	SHREA — hybrid resampling for imbalance + missing data	Oscillator-guided Multiple Imputation Oversampling + Random Undersampling + One-Class SVM/LOF + Bagging	Generic imbalanced benchmark data	Oversample-and-impute, then undersample to form balanced subsets, then one-class classifiers combined via bagging	Two-stage resample → one-class classification → bagging ensemble	Improved performance on imbalanced + incomplete datasets	No test on large real-world fraud/medical data; imputation adds overhead; no direct comparison with SMOTE-ENN
2	DCSHS — dual clustering + stage-wise hybrid sampling	Projective Clustering Combination (PCC) + Stage-wise Hybrid Sampling (SHS) + Cluster Transfer Mapping	Multiple public imbalanced datasets	Creates subsets via projective clustering, finds overlap regions per subset, applies hybrid sampling in overlap zones	Clustering-aware resampling (not just class-ratio aware)	Handles imbalance **and** class overlap together	Solves overlap problem but method is heavy for solo implementation

S.No	Topic	Algorithm	Dataset	Method	Approach	Output / Outcome	Improvement Needed / Research Gap
3	Multiclass imbalance learning survey	Review of resampling + ensembles, binary vs. multiclass	Healthcare, finance, network security, telecom datasets	Comparative analysis; notes most prior work (18/24 papers) only covers binary imbalance	Survey / benchmarking	Confirms binary-class bias in literature	Multiclass imbalance is clearly understudied — strong novelty angle
4	Ensemble learning with gradient resampling	Gradient-based instance-level resampling + ensemble	Cost-sensitive benchmark datasets	Views imbalance at the sample/instance level instead of the class level	Per-sample gradient weighting instead of blanket class resampling	Fine-grained rebalancing	Hard to fully reproduce (gradient-boosting internals); good conceptual gap

S.No	Topic	Algorithm	Dataset	Method	Approach	Output / Outcome	Improvement Needed / Research Gap
5	Imbalanced learning survey (2024)	Spatial Distribution-based Under-sampling (SDUS) + RL-based sample selection	Various	SDUS preserves distribution shape via sphere neighborhoods; ensembles combine diverse resampled subsets via voting/weighted averaging	Distribution-preserving under-sampling + ensemble voting	Retains majority-class structure better than random under-sampling	Voting weights are static — no adaptive weighting by local density
6	SMOTE-IF review (medical multi-class)	SMOTE + Isolation Forest filtering	Medical multi-class datasets	Generates synthetic points via SMOTE, filters noisy synthetic samples with Isolation Forest	Oversample → clean noisy synthetic samples	Reduces synthetic-sample noise problem of vanilla SMOTE	Only tested in medical domain; no fusion with stacking ensembles

S.No	Topic	Algorithm	Dataset	Method	Approach	Output / Outcome	Improvement Needed / Research Gap
7	Comparative ensemble study	EasyEnsemble, XGBoost, Random Forest, AdaBoost	Credit Card Fraud, Phishing Websites	Benchmarks resamplers × classifiers; XGBoost reached F1 = 0.89 / PR-AUC = 0.93, beating EasyEnsemble (F1 = 0.87), confirmed via Wilcoxon test	Systematic benchmarking	Direct performance comparison across techniques	Only compares existing methods — good as a **baseline reference**, proposes nothing new
8	ROS/RUS/hybrid sampling comparison	ROS, RUS, ROS-RUS + Bagging/Boosting	Water Quality Classification dataset	Compares resampler+ensemble combinations systematically	Grid comparison of resamplers × ensembles	Random oversampling slightly outperforms undersampling, but advantage is minimal and dataset-dependent	**Key gap:** best resampler is dataset-dependent — no method proposed to auto-select it

S.No	Topic	Algorithm	Dataset	Method	Approach	Output / Outcome	Improvement Needed / Research Gap
9	Ensemble comparison for fraud detection (2025)	Random Forest, XGBoost, LightGBM, Stacking	IEEE-CIS + other fraud datasets	SMOTE combined with multiple ensemble architectures; stacking reached AUC-ROC up to 0.943	SMOTE + ensemble architecture comparison	Stacking outperforms bagging/single models on most metrics	Resampler choice is static (SMOTE only), not adaptive
10	Stacking + SMOTE / SMOTE-Tomek for fraud	XGBoost + LightGBM + Random Forest (base) → XGBoost (meta-learner)	Kaggle Credit Card Fraud (1:492 ratio)	Stacking with SMOTE and SMOTE-Tomek, evaluated via Recall/F1/AUC-PR/AUC-ROC plus CodeCarbon energy tracking	Two resamplers × stacking ensemble, plus sustainability metric	Recall improved up to 6%, F1 ≈ 0.91–0.92, AUC-PR > 0.90	No dataset outside fraud tested; resampler choice still fixed/manual

Consolidated research gap: Papers 5, 8, 9, and 10 collectively establish that (a) no single resampler dominates across datasets, (b) ensemble/voting weights are typically static, and (c) multiclass imbalance is under-explored. ARS-Stack directly targets all three gaps at once, using only well-tested, reproducible components.

4. Proposed System: ARS-Stack

4.1 Core Idea

1. **Cluster** the minority class using KMeans or DBSCAN to find local neighborhoods.
2. **Score** each cluster for local density / class overlap (borrowing the insight from DC-SHS and the SDUS survey, simplified to a tractable solo-project version).
3. **Select** the best resampling technique *per cluster* — from a small candidate pool: SMOTE, ADASYN, SMOTE-Tomek, SMOTE-ENN — using a simple rule-based or lightweight meta-classifier decision (e.g., high overlap → SMOTE-ENN/Tomek to clean noise; sparse/isolated minority region → SMOTE or ADASYN to emphasize harder points).
4. **Merge** the resampled clusters back into one balanced training set.
5. **Train a stacking ensemble**: Random Forest + XGBoost + LightGBM as base learners, with a meta-learner (Logistic Regression or XGBoost) combining their outputs using confidence-weighted voting instead of static averaging.
6. **Evaluate** against fixed-resampler baselines (plain SMOTE, plain ADASYN, plain SMOTE-Tomek) using F1, AUC-PR, G-mean, and MCC — not raw accuracy, since the data is imbalanced.

4.2 Architecture & MLOps Pipeline

The diagram below shows the complete pipeline, from raw data through the novel adaptive-resampling module, into the stacking ensemble, wrapped in the MLOps/orchestration layer (data versioning, experiment tracking, workflow orchestration, containerization, CI/CD, and monitoring).

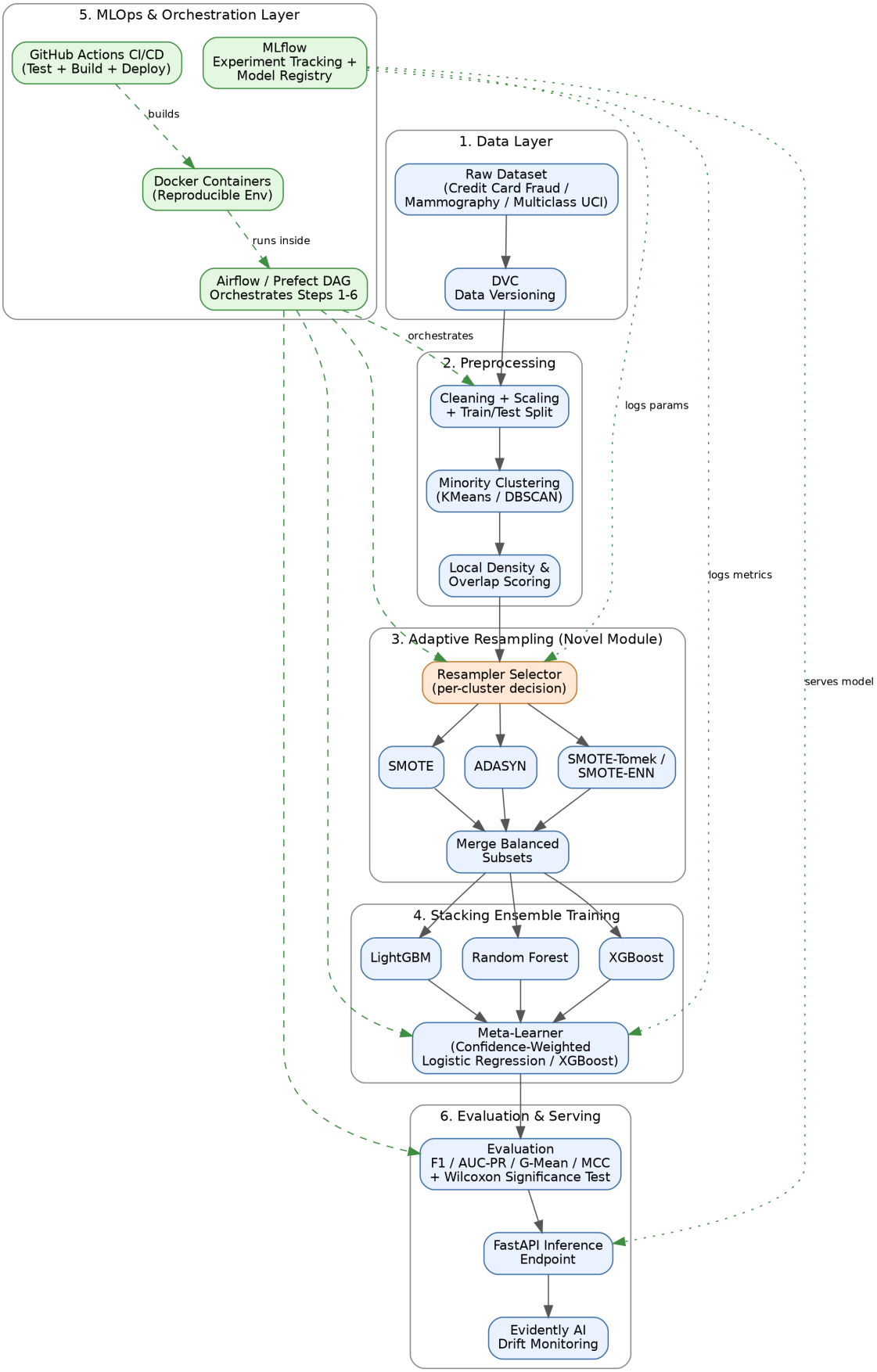


Figure 1: End-to-end ARS-Stack pipeline with MLOps and orchestration layer

5. Datasets

Three datasets are recommended to cover binary extreme imbalance, binary moderate imbalance, and multiclass imbalance — directly targeting the research gap identified in Paper 3 (multiclass under-study).

#	Dataset	Type	Size	Imbalance Ratio	Source	Why chosen
1	Credit Card Fraud Detection (ULB)	Binary, extreme imbalance	284,807 transactions, 492 fraud	≈ 1:578	Kaggle (`mlg-ulb/creditcardfraud`)	Same dataset used in Papers 7, 9, 10 — enables direct, apples-to-apples baseline comparison
2	Mammography Imbalanced Dataset	Binary, moderate-high imbalance	~11,183 samples, ~260 minority	≈ 1:42	Built into `imbalanced-learn` via `fetch_datasets()`	Standard imbalance-learning benchmark, widely cited, medical-adjacent domain, no licensing friction
3	Yeast / Page-Blocks (UCI multiclass)	Multiclass imbalance	Yeast: 1,484 samples, 10 classes (ratio up to 1:92)	Highly skewed across classes	UCI Machine Learning Repository	Fills the explicit multiclass-imbalance research gap noted in Paper 3

All three are public, free, and require no special access — ideal for a solo student with a normal laptop (no GPU required; the entire pipeline runs on CPU with scikit-learn/XGBoost/LightGBM).

6. Algorithms & Tools

6.1 Resampling Techniques (candidate pool for the adaptive selector)

- SMOTE (baseline oversampler)
- ADASYN (adaptive synthetic sampling, emphasizes harder-to-learn minority points)
- SMOTE-Tomek (oversample + remove Tomek links to clean the boundary)
- SMOTE-ENN (oversample + Edited Nearest Neighbors cleaning)

6.2 Clustering / Region Detection

- KMeans (simple, fast, easy to tune `k`)
- DBSCAN (optional, for density-based cluster shapes; useful ablation)

6.3 Base Learners (Ensemble)

- Random Forest (`scikit-learn`)
- XGBoost
- LightGBM

6.4 Meta-Learner

- Logistic Regression (simple, interpretable) — primary choice
- XGBoost as meta-learner — ablation/comparison option (as used in Paper 10)

6.5 Evaluation Metrics

- F1-score (minority class)
- AUC-PR (Precision-Recall AUC — most informative under imbalance)
- G-mean
- Matthews Correlation Coefficient (MCC)
- Wilcoxon signed-rank test for statistical significance (as used in Paper 7)

6.6 Core Libraries

`scikit-learn`, `imbalanced-learn`, `xgboost`, `lightgbm`, `pandas`, `numpy`, `matplotlib`/`seaborn` for plots.

7. MLOps & Orchestration Integration

This is what elevates the project from "just a Jupyter notebook" to a demonstrable, reproducible ML system — valuable both for grading and for a research paper's reproducibility section.

7.1 Components and Their Role

Layer	Tool	Purpose in this project
Version Control	Git + GitHub	Source code history, branching per experiment
Data Versioning	DVC (Data Version Control)	Track dataset versions and preprocessing artifacts alongside Git commits
Experiment Tracking	MLflow	Log every run: which resampler was chosen per cluster, ensemble hyperparameters, and resulting metrics; compare runs side-by-side
Model Registry	MLflow Model Registry	Register the best-performing ARS-Stack model version for deployment
Workflow Orchestration	Apache Airflow (or Prefect, lighter-weight alternative)	Define a DAG: `ingest → preprocess → cluster → adaptive_resample → train_ensemble → evaluate → register_model` — each stage as a task, re-runnable independently
Containerization	Docker	Package the pipeline + dependencies into a reproducible image; avoids "works on my machine" issues, important if reviewers/professors want to re-run it
CI/CD	GitHub Actions	On every push: run unit tests on the resampler-selector logic, lint code, rebuild Docker image
Model Serving	FastAPI	Expose a `/predict` endpoint wrapping the registered MLflow model — lets you demo the project live

Layer	Tool	Purpose in this project
Monitoring (stretch goal)	Evidently AI	Detect data drift if new transaction/patient data is fed post-deployment — optional polish for extra credit / stronger paper

7.2 Why Prefect/Airflow Instead of Just Running Scripts

Manually running notebook cells makes it hard to prove reproducibility to an examiner or reviewer. An orchestrated DAG:

- Makes every pipeline stage independently testable and re-runnable.
- Automatically retries failed stages (e.g., a resampling call that fails on an edge-case cluster with too few points).
- Produces a visual DAG graph you can literally show your professor/examiner during evaluation — strong demonstration value.

7.3 Suggested Repository Structure

```
ars-stack/
  dags/                   # Airflow/Prefect DAG definitions
  src/
    data/                 # ingestion + cleaning
    clustering/           # minority clustering + density scoring
    resampling/           # adaptive resampler selector
    ensemble/             # stacking ensemble training
    evaluation/           # metrics + significance testing
  notebooks/              # exploratory analysis only (not the pipeline itself)
  configs/                # YAML configs (Hydra-style) per experiment
  tests/                  # pytest unit tests for each module
  Dockerfile
  docker-compose.yml      # spins up Airflow + MLflow locally
  requirements.txt
  .github/workflows/ci.yml
  README.md
```

8. Project Timeline (12 Weeks)

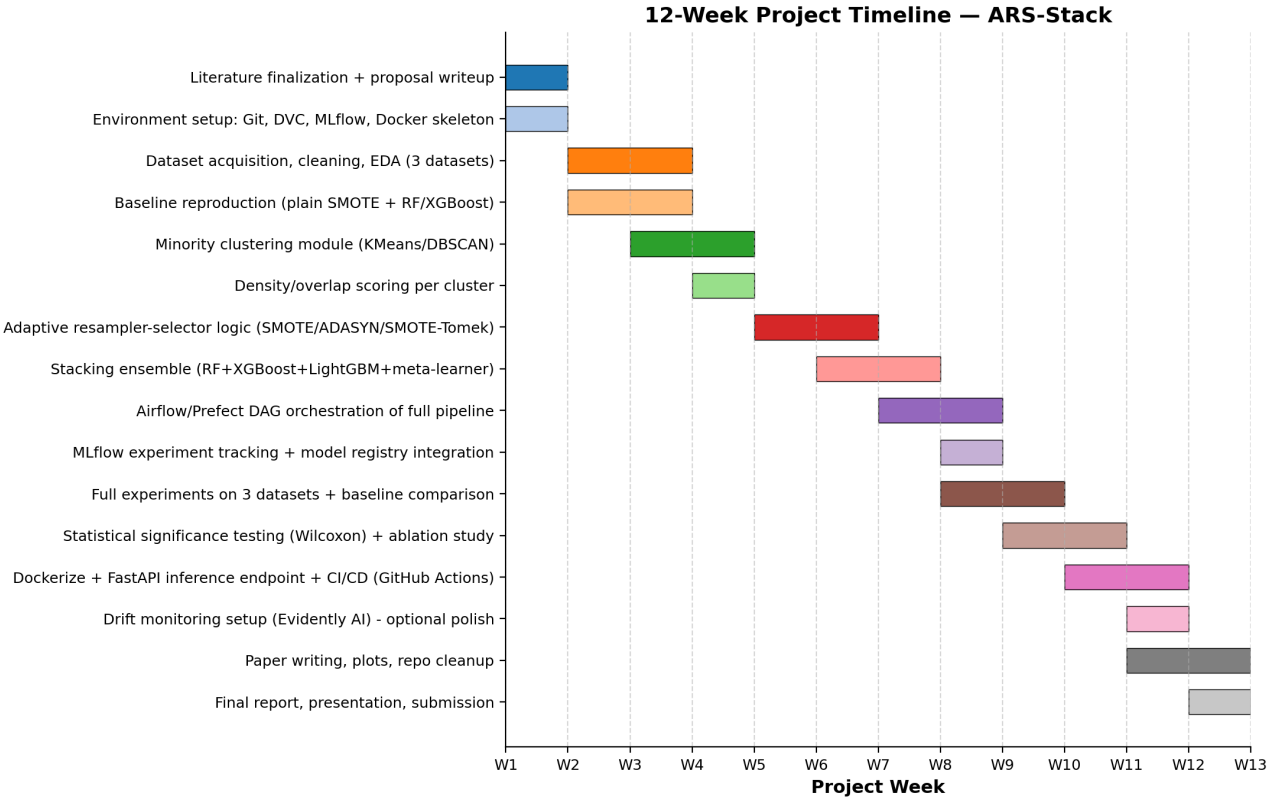


Figure 2: 12-week Gantt timeline for ARS-Stack

8.1 Week-by-Week Breakdown

Week	Milestone
1	Finalize literature review and written proposal; set up Git repo, DVC, MLflow, and Docker skeleton
2	Acquire and clean all 3 datasets; exploratory data analysis; reproduce baseline numbers (plain SMOTE + RF/XGBoost) from Papers 7, 9, 10
3–4	Build minority-clustering module (KMeans/DBSCAN) and local density/overlap scoring
5–6	Build the adaptive resampler-selection logic (SMOTE/ADASYN/SMOTE-Tomek/SMOTE-ENN decision layer)
6–7	Build the stacking ensemble (RF + XGBoost + LightGBM + meta-learner with confidence weighting)

Week	Milestone
7–8	Wire everything into an Airflow/Prefect DAG; integrate MLflow tracking for every run
8–9	Run full experiments across all 3 datasets; compare against fixed-resampler baselines
9	Statistical significance testing (Wilcoxon) + ablation study (with/without adaptive selection, with/without confidence weighting)
10	Dockerize the full pipeline; add FastAPI inference endpoint; set up GitHub Actions CI/CD
11	(Optional polish) Evidently AI drift-monitoring demo; begin paper writing and plot generation
12	Finalize report/paper, clean repository, prepare final presentation and submission

9. Evaluation Plan

1. **Baselines to beat:** plain SMOTE + Random Forest, plain SMOTE + XGBoost, plain SMOTE-Tomek + Stacking (reproducing Papers 7, 9, 10 numbers on the same datasets).
2. **Proposed method:** ARS-Stack (adaptive per-cluster resampler selection + confidence-weighted stacking).
3. **Metrics:** F1 (minority class), AUC-PR, G-mean, MCC — report all four, not just one, since imbalance metrics can disagree.
4. **Significance testing:** Wilcoxon signed-rank test across repeated cross-validation folds, following the approach in Paper 7.
5. **Ablation studies:**
    - ARS-Stack vs. same ensemble with a single fixed resampler (isolates the value of adaptive selection).
    - Confidence-weighted stacking vs. simple averaging (isolates the value of the weighting scheme).
    - KMeans vs. DBSCAN for the clustering step (robustness check).

10. Risk Assessment & Mitigation

Risk	Likelihood	Mitigation
Adaptive resampler-selector adds complexity without clear gain on some datasets	Medium	Report negative/neutral results honestly as part of the ablation — still valid and publishable as an empirical finding
Clustering step unstable on very small minority clusters (e.g., Yeast dataset)	Medium	Set a minimum cluster size threshold; fall back to global SMOTE for undersized clusters
Airflow setup overhead eats into project time	Medium	Use Prefect instead (lighter-weight, pure Python, faster to set up solo) or Airflow's `LocalExecutor` with `docker-compose`
Scope creep (trying to add too many resamplers/classifiers)	High	Freeze the candidate pool (4 resamplers, 3 base learners, 1 meta-learner) after Week 2 — do not expand mid-project
Time lost to environment/dependency issues	Medium	Dockerize environment in Week 1, not Week 10, so the same image is used throughout

11. Expected Contribution & Publication Angle

- **Novelty claim:** "Per-cluster adaptive resampler selection combined with confidence-weighted stacking outperforms static-resampler ensemble baselines across binary-extreme, binary-moderate, and multiclass imbalance settings."
- **Target venues:** student/regional ML conferences, Q3–Q2 journals in applied ML (e.g., venues that previously published Papers 1, 2, 6, 9, 10 above), or a university symposium — realistic within the semester timeframe.
- **Patent angle:** Unlikely to be patentable on its own (algorithmic combination of public techniques), but the **engineered MLOps pipeline artifact** (Dockerized, orchestrated, reproducible) is a strong portfolio/resume asset regardless of publication outcome.
- **Grading angle:** Satisfies "novelty" (new combination + empirical finding), "research paper" (directly writable from the ablation results), and "complex systems engineering" (MLOps integration) requirements in one project.

12. Deliverables Checklist

- ☐ GitHub repository with full source code, tests, and README
- ☐ Dockerized, reproducible pipeline (`docker-compose up` runs the whole thing)
- ☐ Airflow/Prefect DAG screenshot + working orchestration
- ☐ MLflow experiment dashboard with all logged runs
- ☐ FastAPI `/predict` demo endpoint
- ☐ Final results table: ARS-Stack vs. baselines, all 3 datasets, all 4 metrics
- ☐ Ablation study results and plots
- ☐ Written report / draft paper (Introduction, Related Work, Method, Experiments, Results, Conclusion)
- ☐ Final presentation slides

13. References (Literature Reviewed)

1. Baro, "SHREA: A Systematic Hybrid Resampling Ensemble Approach Using One Class Classifier," *Computational Intelligence*, Wiley, 2024.
2. Li, F., Wang, B., Wang, P. et al., "An imbalanced ensemble learning method based on dual clustering and stage-wise hybrid sampling," *Applied Intelligence*, 53, 21167–21191, 2023.
3. "Effectiveness of data resampling and ensemble learning in multiclass imbalance learning," *Artificial Intelligence Review*, Springer, 2025.
4. Zhao, H., Zhao, C., Zhang, X., et al., "An Ensemble Learning Approach with Gradient Resampling for Class-Imbalance Problems," *INFORMS Journal on Computing*, 35(4), 747–763, 2023.
5. "A survey on imbalanced learning: latest research, applications and future directions," *Artificial Intelligence Review*, Springer, 2024.
6. "A review on over-sampling techniques in classification of multi-class imbalanced datasets: insights for medical problems," *Frontiers in Digital Health*, 2024.
7. "A comparative study of ensemble learning techniques for imbalanced classification," *World Journal of Advanced Research and Reviews*, 19(01), 1633–1643, 2023.
8. "Comparison of ensemble hybrid sampling with bagging and boosting machine learning approach for imbalanced data," *Indonesian Journal of Electrical Engineering and Computer Science*, 29(1), 598–608, 2023.
9. "Comparing Ensemble Methods for Credit Card Fraud Detection: A Performance Analysis on Multiple Datasets," Preprints.org, 2025.
10. "STACKING ENSEMBLE, XGBOOST DAN SMOTE UNTUK EFISIENSI ENERGI PADA FRAUD CREDIT CARD," *Jurnal Teknik Informasi dan Komputer*, 2025.

Note: full citation details (DOIs/links) should be re-verified and formatted per your institution's required citation style (IEEE/APA) before submission.